P4DAD: Securing Duplicate Address Detection Using P4

Peng Kuang, Ying Liu, Lin He✉
Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China
Beijing National Research Center for Information Science and Technology (BNRist)

***Abstract*—Duplicate Address Detection (DAD) is an essential part of the Neighbor Discovery Protocol (NDP), which determines whether the IPv6 address of a node conflicts with those of other nodes. Due to the lack of verification of NDP messages, DAD is vulnerable to DoS attacks. Existing solutions suffer from high complexity, need to modify the NDP, or have a single point of failure.**

To solve the above problems, we propose P4DAD, a secure DAD mechanism described by P4. By creating and maintaining binding entries between IPv6 address and link-layer property of host, P4DAD can filter spoofed NDP messages in an in-network manner to prevent DoS attacks on DAD without modifications to the NDP or host stack. We implement a prototype of P4DAD and evaluate it in terms of functionality, performance, and scalability. Evaluation results show that P4DAD can prevent DoS attacks on DAD successfully with negligible overhead, and has satisfactory scalability.

***Index Terms*—Duplicate Address Detection, P4, DoS attack, Neighbor Discover Protocol, IPv6**

I. Introduction

Duplicate Address Detection (DAD) is a crucial procedure in the process of IPv6 [1] address configuration. It allows all nodes located on the same subnet to communicate and to join the network with unique IPv6 addresses. The DAD procedure is based on two Neighbor Discovery Protocol (NDP) [2] messages, namely, Neighbor Solicitation (NS) and Neighbor Advertisement (NA). In DAD, a node sends one or more NS messages, and it will configure the target address only if no NA messages from existing nodes are received. Original DAD assumes that all nodes on a subnet are trustworthy. However, this assumption is hard to achieve in the real network environment. If a malicious node launches Denial-of-Service (DoS) attacks by replying spoofed NA messages continuously when receiving NS messages, the node can not configure any IPv6 addresses.

The above problem is severe due to its denial of a node's access to the network. It will become more severe in the future because more than 20 billion of the Internet of Things (IoT) devices will connect to the Internet by 2020 [3]. These IoT devices have a tendency to use IPv6 address for communication [4], [5] and they still need to use DAD to determine the uniqueness of the IPv6 address [6].

To address this issue, many solutions have been proposed, which can be classified into three categories: target address hiding, NDP message authenticating and NA replying. The first is to use hash encryption or other mechanisms to hide target addresses in NS messages from unintended recipients. The second is to append the Message Authentication Code calculated by hash functions to NDP messages to make it possible for a host to distinguish valid messages from fake ones. The third is to introduce a central node to reply with NA messages uniformly when receiving NS messages. However, the first two solutions often require an update of the host stack due to the modification of the NDP. The third solutions always suffers a single point of failure.

The root of the problem with the first two types of solutions is that they secure the DAD from the host side, and the third type is its dependence on a central controller. Inspired by the data plane programming language P4 [7], we propose P4DAD, which can secure DAD in an in-network manner. The core idea of P4DAD is to filter bogus NA messages of which the target address does not belong to hosts.

P4DAD first uses P4 to monitor the address assignment procedure to create binding entries between a host IPv6 address and a link-layer property of the host's network attachment (e.g., switch port). Then, P4DAD filters NA packets using banding entries to ensure the source address and target address of each NA packet are the same and belong to innocent hosts. P4DAD neither modifies the NDP nor requires a dependency on supportive functionality on hosts, which renders it widely deployable. Moreover, as P4DAD does not introduce hash encryption computation in the host and brings small processing overhead in the switch, it is lightweight. Finally, P4DAD can be deployed on P4 switches directly without the need for a central node, which makes it avoid single points of failure.

In summary, the main contributions of this paper are as follows:

- We propose P4DAD, which is a lightweight, deployable, robust, secure DAD mechanism, can secure DAD without modifications to the NDP or host stack.
- We propose a method allowing P4 switch to configure and populate flow rules locally, avoiding the use of a remote central controller.
- We implement a P4DAD prototype composed of a data plane program and a control plane program, which can prevent Denial-of-Service (DoS) attack on DAD effectively.

✉ Lin He is the corresponding author.

978-1-7281-5089-5/20/$31.00 ©2020 IEEE

TABLE I
COMPARISON OF NS AND NA MESSAGES IN DAD

Message Type	Source Address	Destination Address	Target Address
NS	Unspecified	Multicast address	Tentative address
NA	Interface address	Multicast address	IPv6 address

The remainder of the paper is organized as follows: The related work is reviewed in Section II. Section III describes the problem formulation of P4DAD. P4DAD design is described in Section IV. Section V describes the implementation and evaluation of P4DAD. The discussion is described in Section VI. Finally, Section VII concludes the paper.

II. RELATED WORK

A. DoS Attack on DAD

DAD is an essential procedure during the process of IPv6 address configuration, including manual configuration, Stateless Address Autoconfiguration (SLAAC) [8], and Dynamic Host Configuration Protocol for IPv6 (DHCPv6) [9]. As shown in Table I, in DAD, the source address of NS is unspecified, and the destination address is a solicited-node multicast address where the NS will be multicast. The target address in an NS message is the tentative address which a node wants to configure. Different from NS messages, the source address in an NA message is the interface address of a node, and the destination address is an all-node multicast address, which means NA messages will be broadcast to all nodes on the same subnet. The target address in an NA message is an IPv6 address to which a node performs duplicate detection.

During the process of DAD, a host first multicasts NS messages to verify the uniqueness of an IPv6 address, which it tries to configure. Another host will reply with an NA message when it discovers the IPv6 address is a duplicate. However, it is easy for a malicious host to forge NA messages because there exist no secure mechanisms to verify whether the sender of an NA message owns the IPv6 address or not. For example, as shown in Fig. 1, if a malicious host replies with NA messages for any NS messages in DAD, the victim attached on the same subnet can not configure any addresses.

B. Existing Solutions

1) Hiding Target Addresses: The core idea of this kind of solution is to hide target addresses in NS messages during the process of DAD. DAD-match [10] and pull model [11] embed the hash value of the target address instead of putting it directly in an NS message. Other hosts compute the hash values of their addresses and compare the results with the value in the NS message to determine whether they need to reply with NA messages. Wang et al. [12] propose a mechanism that a host masks a segment of target address, and other hosts compare the unmasked part of the target address and reply with standard NA message if the target address is a duplicate. The host collects all NA messages to determine whether the address is unique or not. Although this kind of solution can mitigate DoS attacks on DAD, they require modification to the NDP, which makes it challenging to deploy in the current networks.

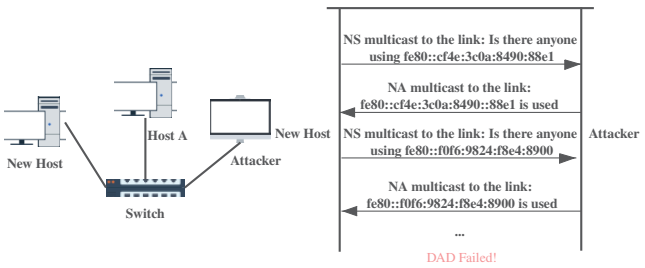

Fig. 1. A DoS Attack on DAD.

2) Authenticating NDP messages: This kind of solution enhances DAD security by appending the Message Authentication Code to NDP messages. Secure-DAD [13] and Trust-ND [14] calculate Message Authentication Code by using SHA-1 and UMAC hash function, respectively. By utilizing this characteristic that malicious hosts are hard to forge Message Authentication Code, benign hosts can distinguish valid messages from fake ones by Message Authentication Code. However, they both generate security issues and introduce extra options into NDP messages, which requires modification to the NDP.

3) Replying with NA Messages Uniformly: The last kind of solutions is to utilize a central, reliable node to reply with NA messages uniformly, and any other NA messages sent by other nodes will be neglected. Nelle et al. [15] use an SDN switch as the NDP proxy which intercepts all NDP messages and forwards them to the SDN controller. As the controller maintains all allocated IPv6 addresses, it replies an NA message if the tentative address is a duplicate. Rule-based mechanism [16] uses a node to store all configured IPv6 addresses. Other nodes must receive its confirmation message to verify whether the tentative address is a duplicate or not. Both of them need a central node to store all IPv6 addresses and thus face a single point of failure problem. Moreover, Rule-based mechanism requires the modification to the NDP.

Table II summarizes the limitations of existing solutions.

III. PROBLEM FORMULATION

A. Design goals

P4DAD aims to enhance DAD security without introducing any modification to the NDP in a lightweight and robust manner, which implies three goals:

TABLE II
LIMITATIONS OF EXISTING SOLUTIONS

Mechanism	Insecure	Heavy weight	Not robust	Hard to deploy
DAD-match [10]		✓		✓
pull model [11]		✓		✓
Wang et al. [12]		✓		✓
Secure-DAD [13]	✓	✓		✓
Trust-ND [14]	✓	✓		✓
Nelle et al. [15]			✓	
Rule-based [16]			✓	✓

- **Lightweight**: P4DAD should not introduce hash encryption computation to hosts or network devices, which will increase computation complexity. Besides, P4DAD should only bring a small performance overhead to switches.
- **Deployable**: P4DAD should not introduce modifications to the NDP or host stack. This means P4DAD should be designed to be purely network-based, which enables network operators to deploy it without a dependency on supportive functionality on hosts.
- **Robust**: P4DAD should not introduce a central node, which requires that P4DAD be able to prevent DoS attacks on DAD in a distributed manner. This ensures that P4DAD can avoid a single point of failure.

B. Threat Model

We assume that hosts connect to switches (In IoT Network, the host is IoT node and the switch is IoT gateway or IoT proxy [6]) directly. Hosts may be malicious and can send arbitrary spoofed NDP messages which may contain spoofed source addresses or target addresses. We also assume that switches are trustworthy and can process received NDP messages correctly.

C. Challenges

To design P4DAD, we mainly face two challenges:

- **Multiple IPv6 Addresses Per Host**: In IPv6 networks, one host usually configures more than one IPv6 address. One is a link-local address, and the others may be assigned by SLAAC or DHCPv6. How to deal with them in DAD for the host is challenging.
- **Auto-Configuration of Flow Rules without Central Controller**: P4 is a data plane programming language that can be used to describe how to parse and handle packets. P4 programs often use flow rules from the control plane to decide which action to take for every packet. Although P4Runtime [17] provides a control plane programming interface to configure and populate flow rules, it needs to maintain complete network topology information, which is likely to face a single point of failure. How to configure and populate flow rules in switches locally is challenging.

IV. P4DAD Design

P4DAD utilizes two key features of P4 to secure DAD: 1) programmable parser to extract target address in NS/NA messages, and 2) stateful memory register to preserve port and IPv6 address binding in data plane directly.

To solve the problem of multiple IPv6 addresses on a single host, P4DAD allows a switch port to correspond to multiple IPv6 addresses. To realize the auto-configuration of flow rules, P4DAD uploads the digest of required information in packets to control plane and notifies it to populate appropriate flow rules in real-time.

A. Overview of P4DAD

On the whole, P4DAD is composed of a data plane and a control plane. As shown in Fig. 2, the data plane contains parser, pipeline, deparser, and register. The parser is responsible for extracting the header of an NS/NA message for processing by pipeline. The pipeline is the core component of the data plane composed of if-else structures and match-action tables. As shown in Table III, the pipeline performs different operations on NS/NA messages for different functions. First, the pipeline verifies the source and target addresses in the header to drop spoofed packets. Then, it sends MAC addresses and ingress ports to control plane for MAC address learning. Next, it queries the target address to determine whether to update the binding entry of IPv6 address and port in the register. Finally, it forwards packets based on the destination MAC address. The control plane is a local controller of the switch, which receives information from the data plane and populates appropriate flow rules promptly.

B. Register Design

To verify the authenticity of the source and target addresses in NS/NA messages. P4DAD utilizes on-switch registers to store the bindings between IPv6 addresses and ports. A key issue is how to calculate the register index. Intuitively, as every port in a switch is connected to a host, a simple solution is to regard a switch port as the register index and set the IPv6 address as the corresponding value. However, every host often has multiple IPv6 addresses. So we have to reserve multiple IPv6 addresses for every switch port in registers. It is assumed that a host has m IPv6 addresses ($m \geq 2$). As $P4_{16}$ [18] only supports multiplication by a power of two currently, m must be a power of two. The calculation of the register index based on the switch port is as follows:

$$index = m \times p + offset, \tag{1}$$

where p is the switch port number and $offset$ is the number of saved IPv6 addresses that correspond to the port p.

To distinguish different IPv6 address states (e.g., tentative, preferred, deprecated) [8], registers also need to store IPv6 address states. As DAD is performed on the same subnet, registers only store a 64-bit IPv6 interface identifier. As shown in Fig. 3, we use two registers to store IPv6 addresses and their states separately.

C. Self-learning Table Building

P4DAD uses three match-action tables, i.e., mac_address_query, target_address_query, and forwarding, to query MAC addresses, target addresses and forward packets, respectively. To avoid using a remote central controller to configure and populate flow rules, P4DAD implements MAC address and IPv6 address self-learning in switches locally.

The key issue of self-learning is how to notify the control plane to populate specified flow rules. Fortunately, $P4_{16}$ provides two methods to solve this problem: packet cloning and packet digest. The former is used to copy the entire content of packets to the control plane. The latter is used to

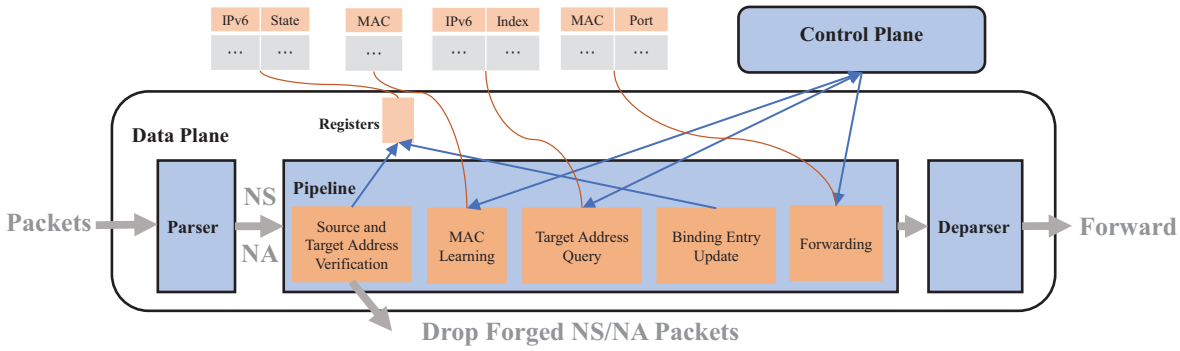

Fig. 2. Design Overview of P4DAD.

TABLE III
DIFFERENT OPERATIONS ON NS/NA MESSAGES FOR DIFFERENT FUNCTIONS

Message Type	Function	Source Verification	Target Verification	MAC Learning	Target Query	Binding Entry Update	Forwarding
NS	DAD			✓	✓	✓	✓
NS	Address Resolve	✓					✓
NS	Unreachability Detection	✓					✓
NA	DAD	✓	✓		✓	✓	✓
NA	Address Resolve	✓			✓		✓
NA	Unreachability Detection	✓			✓		✓
NA	Unsolicited	✓			✓		✓

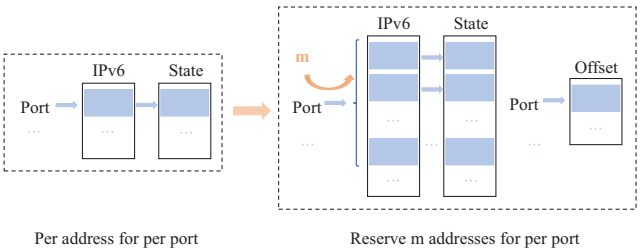

Fig. 3. Register Design for P4DAD.

send the digest of some fields in the packet to the control plane. As we only need to inform the control plane of source MAC address and ingress port from the mac_address_query and forwarding, and IPv6 address and register index from the target_address_query, we prefer to use packet digest in P4DAD.

D. Binding Entry Update

P4DAD creates bindings between IPv6 addresses and ports by listening to the DAD procedure. For NS and NA messages, P4DAD adopts different processing logic as summarized in Algorithm 1. First, P4DAD extracts the target address from the packet header. Then P4DAD determines whether the packet header is NS or NA. If the packet header is NS and the target address is not in the tgr_tbl, P4DAD calculates the register index based on the ingress port (as shown in equation 1). It wraps the IPv6 address and its state in the corresponding register. If the packet header is NA and the target address is in the tgr_tbl, P4DAD acquires the index from the hitting entry and then obtains IPv6 address and its state from the $ipv6_reg$ corresponding to the index. And if the state of IPv6 address is "TENTATIVE", P4DAD resets corresponding $ipv6_reg$.

Algorithm 1 Binding Entry Update for NS/NA

Input: pkt_hdr, tgr_tbl, $ipv6_reg$, "TENTATIVE"
▷ pkt_hdr refers to packet header
▷ tgr_tbl refers to match-action table
▷ $ipv6_reg$ refers to register
▷ "TENTATIVE" refers to IPv6 address state
Extract $targetAddr$ from pkt_hdr
if pkt_hdr is NS **then**
  if $targetAddr$ misses tgr_tbl **then**
    Extract $ingressPort$ from pkt_hdr
    Calculate $index$ according to $ingressPort$
    $ipv6_reg$[index] ← $targetAddr$,"TENTATIVE"
  end if
else if pkt_hdr is NA **then**
  if $targetAddr$ hits tgr_tbl **then**
    Extract $index$ from hitting entry
    $ipv6$, $state$ ← $ipv6_reg[index]$
    if $state$ == "TENTATIVE" **then**
      reset $ipv6_reg[index]$
    end if
  end if
end if

E. P4DAD Workflow

We illustrate the P4DAD workflow in a simple network topology, which contains two P4 switches, as shown in Fig. 4. Host 1 and Host 2 (malicious host) have configured IPv6 addresses, respectively, So the binding entries of IPv6_1 and

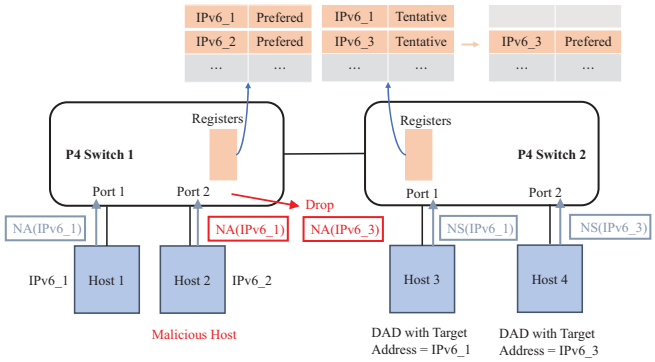

Fig. 4. Workflow of the P4DAD.

IPv6_2 have been established in P4 Switch 1. Host 3 and Host 4 are conducting the DAD process. Host 3 tries to configure the same IPv6 address as that of Host 1. When receiving NS packets sent by Host 3 and Host 4, P4 Switch 2 will establish the binding entries of IPv6_1 and IPv6_3, and set the corresponding IPv6 address state to "Tentative". Host 1 will reply an NA packet with IPv6_1 as the source and target address. However, the malicious Host 2 will send two spoofed NA packets with IPv6_1 and IPv6_3 as the target address, respectively. When spoofed NA packets are received by P4 Switch 1, they will be filtered directly, which means the DoS attack on DAD is prevented effectively. When P4 Switch 2 receives the NA packet sent by Host 1, the binding entry of IPv6_1 will be cleared. When no NA packets about IPv6_3 are received by P4 Switch 2 at a certain time, the corresponding IPv6 address state will be set to "Preferred".

V. Implementation and Evaluation

In this section, we describe the implementation of P4DAD and evaluate its effectiveness and performance. First, we illustrate the P4DAD implementation. Then, we evaluate the effectiveness of P4DAD on DoS attacks prevention. Next, we compare the processing time of NS/NA messages between P4DAD and baseline and measure the communication time between the data plane and the control plane to evaluate the performance of P4DAD. Finally, we evaluate the scalability of P4DAD by measuring the P4 switch memory consumption when increasing the number of hosts. Our experiments are conducted on a Dell OptiPlex 7050 server with one Intel(R) Core(TM) i7-7700 CPUs and 7840 MB RAM.

A. P4DAD Implementation

The P4DAD prototype is composed of a data plane program and a control plane program. The data plane program is implemented by $P4_{16}$ [18], which can be compiled directly on bmv2 [19]. The control plane program is written in Python, which communicates with the data plane by notifications-addr [19].

B. Functional Test

We build a simple topology where two virtual machines (victim and attacker) installed with the parasite6 tool are connected to a P4 software switch implemented by bmv2 [19]. The attacker sends spoofed NS/NA messages randomly, and the victim sends standard NS/NA messages randomly. We count the number of packets received and filtered by P4DAD, sent by the attacker and the victim separately, as shown in Table IV. The number of packets filtered by P4DAD is equal to that sent by the attacker, which means P4DAD can recognize and filter spoofed NS/NA messages effectively.

TABLE IV
Statistics of NS/NA Packets Using P4DAD

Type	Receiving	Filtering	Attacker Sending	Victim Sending
NS	1179	887	887	292
NA	1172	747	747	425

C. Performance Test

We select the performance of original NDP (no modification to switch) as the baseline and compare the processing time of NS/NA messages between P4DAD and the baseline. To illustrate conveniently, we denote NS-For-DAD and NS-For-Others as NS messages used to handle DAD and other NDP functions, respectively. As shown in Fig. 5, the processing time of P4DAD for NS-for-DAD and NA-for-DAD is greater than the baseline, and the processing time of P4DAD for NS-for-Others and NA-for-Others is very close to the baseline. The reason is that NS-for-DAD and NA-for-DAD have three or four more operations than baseline, and NS-for-Others and NA-for-Others have one or two more operations than baseline, as shown in Table III.

Moreover, we measure the time from the beginning of the data plane digesting packet information to the end of the control plane populating flow rules. We conduct multiple testing, and the average of the time is 4.126 milliseconds.

Commonly, DAD only appears on the process of address configuration, and the communication between the control plane and the data plane only needs to be carried out in the process of DAD. This means most NS/NA messages are used for other NDP functions (e.g., address resolve, unreachability detection), which has processing time close to the baseline. Therefore, the processing overhead brought by P4DAD is negligible.

D. Scalability Test

We measure the memory consumption of the P4 switch to evaluate the scalability of P4DAD when increasing the number of hosts. The memory of P4DAD consists of two parts: register memory and match-action table (MAT) memory. We measure both of them and their sum, respectively. As shown in Fig. 6, the memory consumption of P4DAD increases linearly with the number of hosts when m (See Equation 1) is equal to 2 or 4. Besides, We theoretically calculate the register memory reg_mem and MAT memory mat_mem as follows:

$$\begin{aligned} reg_mem &= 9 \times m \times h \ (bytes) \\ mat_mem &= (10 \times m + 14) \times h \ (bytes) \end{aligned} \tag{2}$$

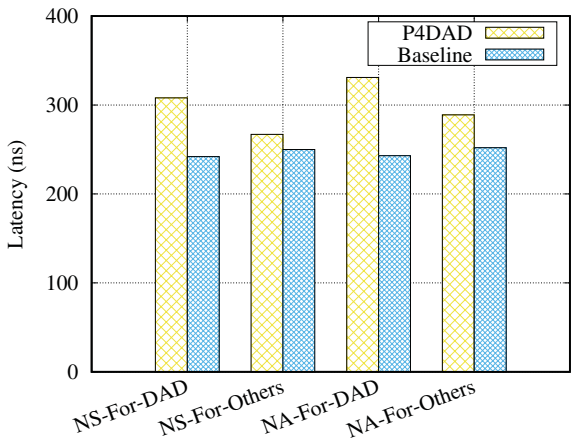

Fig. 5. NS/NA Message Processing Time on the Switch.

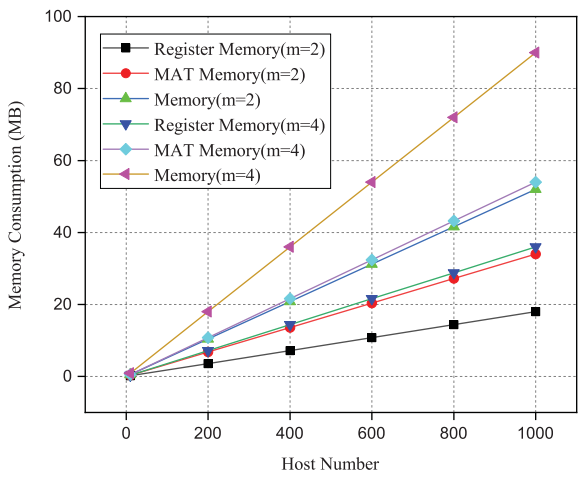

Fig. 6. Memory Consumption Varies as the Number of Hosts Increases.

As shown in Equation 2, at a particular value of m, both register memory and MAT memory are linear with respect to the number of hosts, which is consistent with the previous measurements. In conclusion, both experimental measurements and theoretical analysis show that P4DAD has satisfactory scalability.

VI. Discussion

In this section, we discuss the impact of m (as shown in Equation 1) on register utilization and then evaluate the relationship between register memory assumption and register utilization. The calculation of the register utilization u is as follows:

$$u = \frac{i}{m}, (h \leq m) \tag{3}$$

where i is the number of IPv6 addresses that a host owns. We assume that SLAAC [8] assigns two global IPv6 addresses to the host and DHCPv6 [9] assigns a global IPv6 address to the host. As shown in Table V, we find m = 4 performs best in SLAAC and SLAAC + DHCPv6, and m = 2 performs best in DHCPv6.

TABLE V
Impact of m on register utilization

m	SLAAC	DHCPv6	SLAAC + DHCPv6
2	✗	100%	✗
4	75%	50%	100%
8	37.5%	25%	50%

The calculation of the register memory assumption reg_mem is as follows:

$$reg_mem = \frac{h \times i}{u} \times unit \tag{4}$$

where h is the number of hosts, $unit$ is the register memory size that an IPv6 address occupies (In our example, the size of $unit$ is 72 bytes, with 64 bytes for storing IPv6 address and 8 bytes for storing IPv6 address state), and u and i are shown in equation 3. As shown in equation 4, under the same number of hosts and a certain number of IPv6 addresses assigned by each host, with the increase of register utilization, the register resources consumed by the switch gradually decrease.

VII. Conclusion

In this paper, we propose P4DAD, a lightweight, deployable, robust, secure DAD mechanism. By introducing P4, P4DAD secures NDP messages in the network without the modification of the NDP or host stack. Also, no extra calculations are introduced to the host. By storing used IPv6 addresses locally and utilizing packet digests, P4DAD does not need to rely on a central controller, avoiding single points of failure. To the best of our knowledge, P4DAD is the first secure DAD mechanism described by P4. We implement P4DAD prototype on bmv2 [19]. Our evaluation shows P4DAD can prevent DoS attacks on DAD effectively with negligible overhead, and has satisfactory scalability.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (Grant No. 61772307), and National Key R&D Program of China (Grant No. 2018YFB1800405, 2018YFB1800402, and 2017YFB0801701).

References

[1] D. S. E. Deering and B. Hinden, "Internet Protocol, Version 6 (IPv6) Specification," RFC 8200, Jul. 2017.
[2] W. A. Simpson, D. T. Narten, E. Nordmark, and H. Soliman, "Neighbor Discovery for IP version 6 (IPv6)," RFC 4861, Sep. 2007.
[3] "The 2019 State of IoT Report," Website, https://www.particle.io/solutions/2019-state-of-iot-report/.
[4] H. Lamaazi, N. Benamar, A. J. Jara, L. Ladid, and D. El Ouadghiri, "Challenges of the Internet of Things: IPv6 and Network Management," in *2014 Eighth International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*. IEEE, 2014, pp. 328–333.

[5] A. J. Jara, D. Fernandez, P. Lopez, M. A. Zamora, and A. F. Skarmeta, "Lightweight Mobile IPv6: A Mobility Protocol for Enabling Transparent IPv6 Mobility in the Internet of Things," in *2013 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2013, pp. 2791–2797.

[6] T. Savolainen, J. Soininen, and B. Silverajan, "IPv6 Addressing Strategies For IoT," *IEEE Sensors Journal*, vol. 13, no. 10, pp. 3511–3519, 2013.

[7] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[8] D. T. Narten, T. Jinmei, and D. S. Thomson, "IPv6 Stateless Address Autoconfiguration," RFC 4862, Sep. 2007.

[9] T. Mrugalski, M. Siodelski, B. Volz, A. Yourtchenko, M. Richardson, S. Jiang, T. Lemon, and T. Winters, "Dynamic Host Configuration Protocol for IPv6 (DHCPv6)," RFC 8415, Nov. 2018.

[10] A. K. Al-Ani, M. Anbar, S. Manickam, and A. Al-Ani, "DAD-Match: Technique to Prevent DoS Attack on Duplicate Address Detection Process in IPv6 Link-local Network," *JCM*, vol. 13, no. 6, pp. 317–324, 2018.

[11] G. Yao, J. Bi, S. Wang, Y. Zhang, and Y. Li, "A Pull Model IPv6 Duplicate Address Detection," in *The 35th Annual IEEE Conference on Local Computer Networks, LCN 2010, 10-14 October 2010, Denver, Colorado, USA, Proceedings*, 2010, pp. 372–375.

[12] X. Wang, H. Cheng, and Y. Yao, "Addressing With an Improved DAD for 6LoWPAN," *IEEE Communications Letters*, vol. 20, no. 1, pp. 73–76, 2016.

[13] S. U. Rehman and S. Manickam, "Improved Mechanism to Prevent Denial of Service Attack in IPv6 Duplicate Address Detection Process," *Int. J. Adv. Comput. Sci. Appl*, vol. 8, no. 2, pp. 63–70, 2017.

[14] S. Praptodiyono, I. H. Hasbullah, M. M. Kadhum, C. Y. Wey, R. K. Murugesan, and A. Osman, "Securing Duplicate Address Detection on IPv6 Using Distributed Trust Mechanism," *Int J Simulation—Systems, Sci Technol*, vol. 17, no. 26, 2016.

[15] D. Nelle and T. Scheffler, "Securing IPv6 Neighbor Discovery and SLAAC in Access Networks Through SDN," in *Proceedings of the Applied Networking Research Workshop, ANRW 2019, Montreal, Quebec, Canada, July 22, 2019.*, 2019, pp. 23–29.

[16] S. U. Rehman and S. Manickam, "Rule-based Mechanism to Detect Denial of Service (DoS) Attacks on Duplicate Address Detection Process in IPv6 Link Local Communication," in *2015 4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)(Trends and Future Directions)*. IEEE, 2015, pp. 1–6.

[17] "P4Runtime," Website, https://github.com/p4lang/p4runtime.

[18] M. Budiu and C. Dodd, "The P416 Programming Language," *SIGOPS Oper. Syst. Rev.*, vol. 51, no. 1, pp. 5–14, Sep. 2017.

[19] "The Second Version of the P4 Software Switch, BMv2," Website, https://github.com/p4lang/behavioral-model.